

Algorithm Engineering Notes

Academic Year 2026/2027

Contents

1	Algorithm Design, Computational Models, and Memory Hierarchy	1
1.1	Formal Definition of an Algorithm	1
1.1.1	Exact Algorithms, Artificial Intelligence, and Heuristics	2
1.2	Efficiency Measurement and Asymptotic Analysis	3
1.2.1	Why Not Count the Exact Number of Instructions? . .	3
1.2.2	Why Prefer Worst-Case Analysis?	3
1.3	Computational Models: RAM and Real Hardware	4
1.3.1	The Random Access Machine Model	4
1.3.2	Real Hardware: Memory Hierarchy and the Memory Wall	4
1.3.3	When the RAM Model Is Not Enough	5
1.4	Locality and the Two-Level Model	7
1.4.1	The Two-Level Memory Model	7
1.5	Worked Example: Linear Array Scan	8
1.5.1	Analysis in the Classical RAM Model	8
1.5.2	Analysis in the Two-Level Model	9
1.5.3	Linear Access versus Sparse Access	9
2	Strided Access, B-Trees, and the Impact of Virtual Paging	10
2.1	Non-Sequential Access: The $A_{s,b}$ Algorithm	10
2.1.1	Coprimality Condition	11
2.1.2	I/O Cost	11
2.2	Algorithmic Limits and Hardware Acceleration	11
2.3	Searching Ordered Sets: Binary Search versus B-Trees	13
2.3.1	Binary Search in Secondary Memory	13
2.3.2	B-Trees and B ⁺ -Trees	13
2.3.3	Quantitative Comparison	14
2.4	I/O-Aware Algorithms and Virtual Paging	14
2.4.1	Paging with an Oversized Working Set	15
2.4.2	Numerical Thrashing Example	16
2.5	Key Takeaways	16

3	Permutation, Sorting, and External Memory Merge Sort	17
3.1	The Permutation vs. Sorting Problem	17
3.1.1	Problem Formulation	17
3.1.2	Naive Implementation on the RAM Model	18
3.1.3	Simulation Principle (RAM to Two-Level Memory) . .	19
3.1.4	The Relationship Between Sorting and Permutation . .	19
3.2	Permutation Complexity in External Memory	21
3.2.1	Corner Case Analysis	21
3.2.2	Numerical Verification on Realistic Architectural Pa- rameters	23
3.3	External Memory Binary Merge Sort	24
3.3.1	Buffer Architecture in the Merge Procedure	24
3.3.2	Recursion Tree Complexity	25
3.3.3	Sub-optimality and Bottlenecks	26

Chapter 1

Algorithm Design, Computational Models, and Memory Hierarchy

Lecture 1

Introduction, computational models, and memory hierarchy

Date: September 15, 2026

1.1 Formal Definition of an Algorithm

An *algorithm* is a finite and ordered sequence of unambiguous, elementary instructions that, starting from a specified set of input data, produces a corresponding set of output data within a finite amount of time.

In the classical formulation by Donald Knuth in *The Art of Computer Programming* [1], an algorithm must satisfy five fundamental properties:

Finiteness. The algorithm must always halt after a finite number of computation steps for every admissible input configuration.

Definiteness. Each instruction must be stated clearly, precisely, and without ambiguity; the next execution state must be uniquely determined.

Input. The algorithm receives zero or more quantities specified from outside before execution begins.

Output. The algorithm produces one or more output quantities that have a specified relationship with the input data. The output must be meaningful for the problem being solved.

Effectiveness. Every prescribed operation must be elementary and physically executable in finite time using a bounded amount of resources, such as paper and pencil or a sequential machine.

1.1.1 Exact Algorithms, Artificial Intelligence, and Heuristics

A classical exact algorithm is distinguished from an AI or heuristic method by the guarantees it provides and by how it explores the space of decisions.

Table 1.1: Exact algorithms compared with AI and heuristic methods.

Dimension	Classical exact algorithm	AI or heuristic method
Correctness guarantee	Produces a correct solution for every admissible instance; optimality is guaranteed when the problem specification asks for an optimum.	May prioritize empirical quality or practical efficiency and may lack a universal guarantee for every instance.
Termination	Can be proved in advance using potential functions or loop variants.	Often tied to heuristic stopping criteria, such as a maximum number of epochs or gradient convergence.
Decision space	Determined by a precisely specified procedure, which may be deterministic or randomized.	Explored through heuristic search, statistical inference, or loss-function optimization.

Heuristic programming does not seek a formal proof of absolute optimality over every admissible instance. Instead, it seeks an efficient trade-off for computationally intractable problems, such as NP-complete or NP-hard problems.

1.2 Efficiency Measurement and Asymptotic Analysis

1.2.1 Why Not Count the Exact Number of Instructions?

Defining the exact running cost $T_A(n)$ as the actual number of clock cycles or elementary instructions presents several difficulties:

1. **Architecture dependence:** the result would depend on the specific CPU, instruction set architecture (ISA), and clock frequency.
2. **Toolchain dependence:** it would vary with the compiler, optimization level, and operating system.
3. **Lack of generality:** it would obscure the intrinsic quality of the algorithmic idea.

For this reason, algorithm analysis uses asymptotic notation. It abstracts away implementation details and multiplicative constants, and studies the growth rate of the cost function as the input size tends to infinity: $n \rightarrow \infty$.

Big- $\mathcal{O}$ (asymptotic upper bound):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, n_0 > 0 \text{ such that} \\ 0 \leq f(n) \leq c g(n), \quad \forall n \geq n_0.$$

Big- Ω (asymptotic lower bound):

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 > 0 \text{ such that} \\ 0 \leq c g(n) \leq f(n), \quad \forall n \geq n_0.$$

Big- Θ (tight asymptotic bound):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Why Prefer Worst-Case Analysis?

- **Absolute guarantee:** an upper bound guarantees that the algorithm will never take more time than the theoretical bound. This is essential for real-time and mission-critical systems.
- **Simple formal derivation:** worst-case analysis does not require probabilistic assumptions about the input distribution, which are often needed for average-case analysis.

1.3 Computational Models: RAM and Real Hardware

An algorithm is not executed in an abstract vacuum: it runs on a machine with processors, memories, and data-transfer mechanisms. A computational model specifies which machine details are relevant to the analysis and which are deliberately ignored. The RAM model is the standard starting point; real hardware then explains when that abstraction becomes insufficient.

1.3.1 The Random Access Machine Model

The Random Access Machine (RAM) model is a sequential, word-based abstraction inspired by the von Neumann architecture. A RAM consists of a processor, a finite program, and an addressable memory of words. Its usual assumptions are:

- **Sequential execution:** one processor executes one elementary instruction at a time.
- **Word-sized data:** arithmetic, comparison, assignment, and address calculation on a constant number of words take constant time.
- **Uniform memory access:** reading or writing any address costs $\mathcal{O}(1)$, independently of the address and of the access history.
- **Sufficient memory:** the input and the data structures fit in the modeled address space; the cost of allocating the space is analyzed separately when it matters.

The RAM model is useful because it makes algorithmic ideas comparable. For example, a loop that examines n array entries performs $\Theta(n)$ RAM operations, regardless of the particular processor used to run it. The model does not claim that a real machine has infinite memory or truly constant latency; it states which costs are being abstracted away.

1.3.2 Real Hardware: Memory Hierarchy and the Memory Wall

On a modern machine, the processor is much faster than the memories that hold large datasets. This growing gap between computation and data movement is known as the *memory wall*. Moreover, local memory is not a

single uniform resource: it is organized into levels with different capacities, latencies, and transfer granularities.

A simplified hierarchy, ordered from the fastest and smallest level to the slowest and largest one, is:

CPU registers $\longrightarrow$ L1 cache $\longrightarrow$ L2 cache $\longrightarrow$ L3 cache $\longrightarrow$ DRAM
 $\longrightarrow$ NVMe/SSD/disk $\longrightarrow$ remote storage / cloud.

Registers and caches are part of the processor’s fast memory system; DRAM is the usual main memory; SSDs and disks provide persistent storage. Remote storage and cloud services are included here only as a broader data-access hierarchy, because they are reached through a network rather than through the local memory bus.

Figure 1.1 describes the main local-memory levels: each lower layer trades lower access speed for greater capacity.

Moving downward in Figure 1.1, capacity generally grows while latency and transfer cost grow as well. The exact ratios depend on the hardware and workload, so values such as 100–200 times for DRAM relative to L1, or 10^4 – 10^6 times for disk relative to L1, should be read as order-of-magnitude examples rather than universal constants.

1.3.3 When the RAM Model Is Not Enough

The RAM model is a good first approximation when the computation is mostly sequential, the working set fits in main memory, and the cost of data movement is not the dominant factor. A different model is preferable when one of the following effects controls performance:

GPU computing. SIMT (Single Instruction, Multiple Threads) execution uses many processing cores and distinguishes shared, local, and global memory spaces.

External memory. In out-of-core computation, the dataset does not fit in fast memory, so block transfers and I/O operations dominate the cost.

Parallel and distributed computation. Several processors or machines execute simultaneously, and synchronization and communication become explicit costs.

Quantum computing. Qubits, superposition, entanglement, and unitary gates require a model based on quantum operations rather than RAM words.

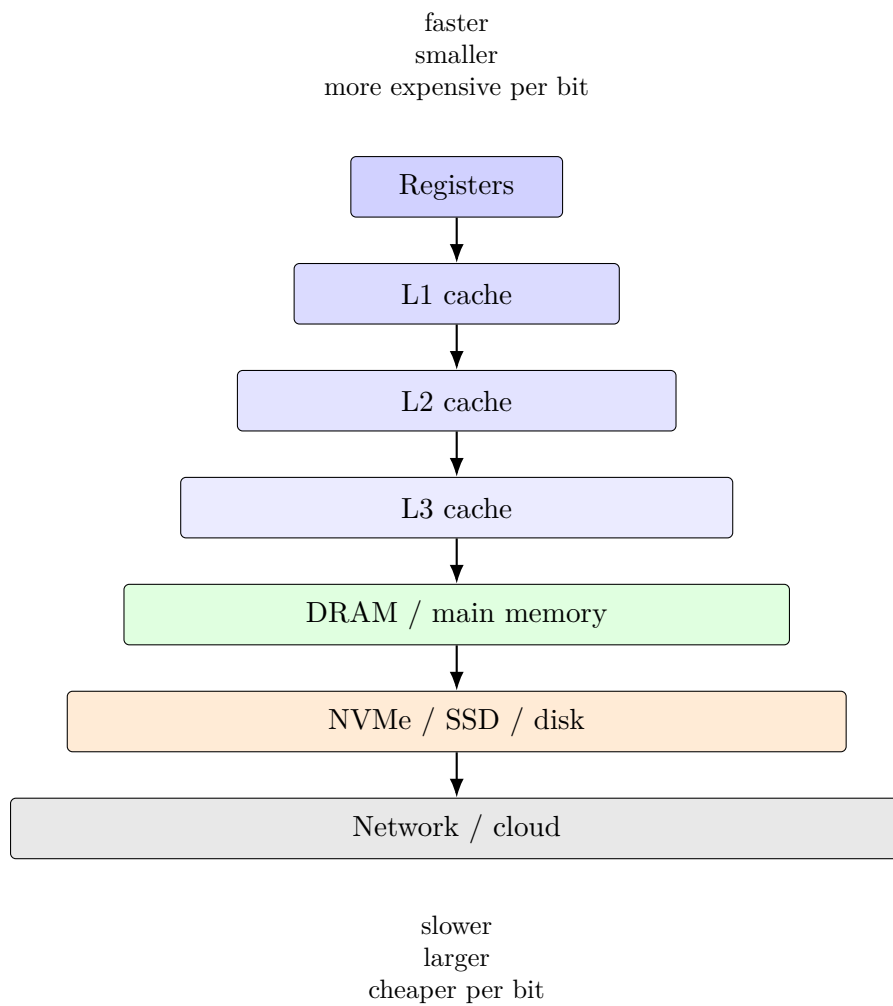


Figure 1.1: A hierarchical view of memory and storage.

The engineering lesson is therefore not to discard the RAM model, but to use the simplest model that still captures the bottleneck of the target system.

1.4 Locality and the Two-Level Model

Modern systems exploit two statistical properties of real programs:

1. **Temporal locality:** if a memory location is referenced at time t , it is likely to be referenced again soon, at $t + \Delta t$.
2. **Spatial locality:** if a memory location x is referenced, nearby locations such as $x + 1$, $x + 2$, and so on are likely to be referenced soon.

1.4.1 The Two-Level Memory Model

When processing gigabytes or terabytes of data, the multilevel hierarchy can be abstracted as a two-level model, associated with the external-memory model of Aggarwal and Vitter [2], also known as the Disk Access Machine model.

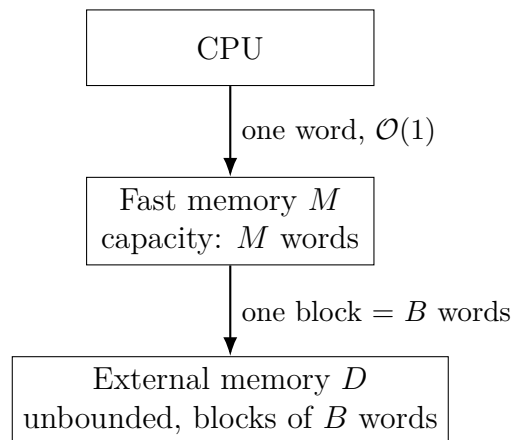


Figure 1.2: Two-level memory model with fast memory M and external memory D .

Figure 1.2 describes the two-level abstraction: the CPU operates on words in fast memory M , while transfers between M and the external memory D occur in blocks of B words.

Model Parameters

N : total number of problem elements to process.

M : size of internal fast memory, measured as the maximum number of elements that can be held simultaneously.

B : size of the block transferred in one I/O operation, with $M \geq 2B$.

Cost Metrics

1. **I/O complexity (primary metric)**: number of transfers of blocks of size B between slow and fast memory.
2. **CPU time (secondary metric)**: number of logical and arithmetic operations performed after data have been loaded into M .
3. **Space**: number of blocks allocated on secondary storage.

This abstraction is fractal: it can model both RAM–disk transfers and L3-cache–RAM transfers by redefining the meaning of M and B .

1.5 Worked Example: Linear Array Scan

Consider a contiguous array of N integers scanned in sequence, for example to compute the sum $\sum_{i=1}^N A[i]$.

Listing 1.1: Sequential array scan

```
1 long sum = 0;
2 for (int i = 0; i < N; i++) {
3     sum += A[i];
4 }
```

1.5.1 Analysis in the Classical RAM Model

- Memory accesses: N .
- Addition instructions: N .
- Asymptotic computation time: $\Theta(N)$.

1.5.2 Analysis in the Two-Level Model

Assume that the secondary memory stores array A consecutively in blocks of size B and that $A[0]$ is aligned with a block boundary. Without this alignment assumption, the scan may require at most one additional block transfer, without changing the asymptotic bound:

1. At the first access to $A[0]$, a miss occurs. The system transfers the block containing $A[0], \dots, A[B-1]$ into fast memory with one I/O.
2. The following $B-1$ accesses find their data already in fast memory, requiring no additional I/O. This exploits spatial locality.
3. A new I/O transfer is needed only when the index crosses a block boundary, that is, once every B elements.

Therefore, the total I/O cost of a linear scan is

$$\text{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ I/O operations.}$$

1.5.3 Linear Access versus Sparse Access

- **Sequential access (high spatial locality):** reading N elements requires N/B I/O operations. Since B typically ranges from 64 bytes for a cache line to hundreds of kilobytes or megabytes for pages and disk blocks, the speedup over the naive model can be a factor of approximately B .
- **Random access or stride $k > B$ (no spatial locality):** every requested element may reside in a different block, causing N independent I/O operations. The cost degrades to $\Theta(N)$ I/Os, which is asymptotically slower by a factor of B .

Chapter 2

Strided Access, B-Trees, and the Impact of Virtual Paging

Lecture 2

Strided access, binary search versus B-Trees, and virtual paging

Date: September 16, 2026

This chapter studies how an algorithm's access pattern interacts with the transfer granularity of a memory system. The same arithmetic work can have very different costs depending on whether accesses preserve spatial locality, match the block size, or exceed the capacity of fast memory.

2.1 Non-Sequential Access: The $A_{s,b}$ Algorithm

Let S be an array containing n elements. The storage system transfers data in physical blocks of size B (a disk block or cache line). We also define a logical block size b , where $b \leq B$ and b divides B . The corresponding numbers of logical and physical blocks are

$$N' = \frac{n}{b}, \quad N_{\text{phys}} = \frac{n}{B}.$$

The algorithm $A_{s,b}$ computes the sum of the elements by visiting logical blocks in a stride pattern. The logical block visited at step j is

$$\text{block}(j) = (js) \bmod N', \quad j = 0, 1, \dots, N' - 1.$$

2.1.1 Coprimality Condition

The sequence visits every logical block exactly once if and only if

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In that case, multiplication by s is a permutation of the residues modulo N' . If the greatest common divisor is $d > 1$, the sequence repeats after $N'/d = n/(bd)$ distinct logical blocks and therefore covers only a fraction of the array.

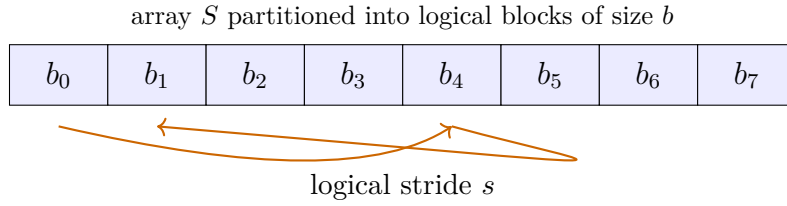


Figure 2.1: A modular stride visits logical blocks in a non-sequential order.

Figure 2.1 illustrates the logical access order. The physical cost depends on how this order maps to blocks of size B , rather than only on the number of arithmetic operations.

2.1.2 I/O Cost

A useful first approximation compares the physical distance between successive logical accesses, sb , with the physical block size B .

The bounds in Table 2.1 describe the usual worst-case or capacity-miss regime. Exact constants depend on alignment, replacement policy, and the interaction between the modular sequence and physical blocks. In the last case, the cost is a factor of roughly B worse than a linear scan, which can exploit every transferred element.

2.2 Algorithmic Limits and Hardware Acceleration

Fix a maximum admissible execution time T_0 and assume that hardware or parallelism accelerates the computation by a factor K . The accelerated machine can therefore afford an original running time of

$$\frac{T(n)}{K} = T_0 \quad \Longleftrightarrow \quad T(n) = KT_0.$$

Table 2.1: Approximate I/O behavior of the strided access pattern.

Condition	Locality and behavior	I/O cost
$sb < B$	Several logical accesses can share one physical block; partial spatial locality is preserved.	$\Theta(n/B)$
$sb \geq B$	Successive accesses typically fall on different physical blocks. If $M < n$, the working set cannot remain entirely in fast memory.	$\Theta(n/b)$
$b = 1$ and $s \geq B$	Each logical access is one element and the stride destroys the benefit of block transfers.	$\Theta(n)$

The effect on the largest solvable input depends on the growth rate of T .

Table 2.2: Effect of a K -fold hardware acceleration on solvable input size.

Algorithm	Complexity	Base hardware	Accelerated hardware	Effective gain
A_1	cn	$n_1 = T_0/c$	$n'_1 = Kn_1$	Multiplicative, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n'_2 = \sqrt{K} n_2$	Sublinear, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n'_3 = n_3 + \log_2 K$	Additive, $+\log_2 K$

Figure 2.2 shows why hardware improvements do not remove an asymptotically inefficient algorithm. For an exponential algorithm, even a 1000-fold speedup increases the solvable input size by only $\log_2(1000) \approx 10$ elements.

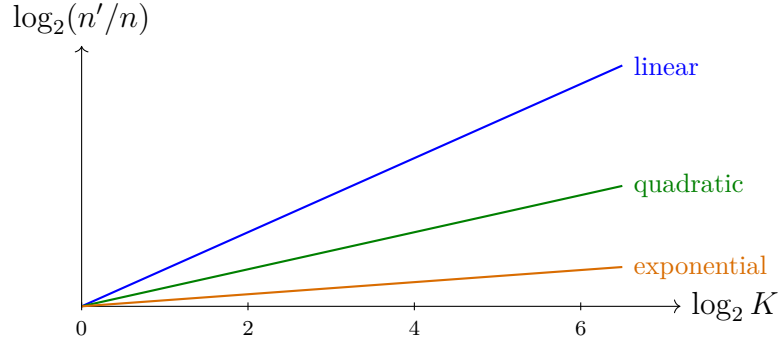


Figure 2.2: Relative benefit of hardware acceleration for representative growth rates.

2.3 Searching Ordered Sets: Binary Search versus B-Trees

2.3.1 Binary Search in Secondary Memory

For a sorted sequence of n keys stored in blocks of size B , binary search performs $\Theta(\log_2 n)$ comparisons on the CPU. Its I/O cost is larger than its comparison count suggests: during the first iterations, the examined positions are separated by $n/2$, $n/4$, and so on. Until the search interval shrinks to at most B elements, each probe can address a different physical block. Thus,

$$\text{I/O}_{\text{binary search}} \approx \log_2 n - \log_2 B = \log_2 \left(\frac{n}{B} \right).$$

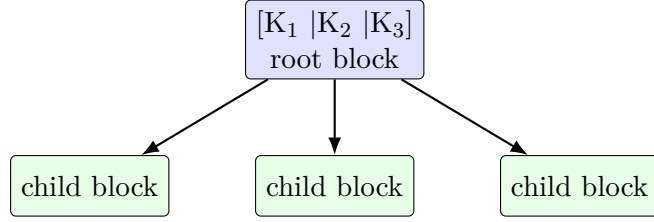
The approximation ignores alignment and the possibility that a block remains cached, but captures the dominant external-memory behavior.

2.3.2 B-Trees and B⁺-Trees

A B-Tree chooses the node size to match one physical block. Each node stores $\Theta(B)$ keys and child pointers, so its fan-out is $\Theta(B)$. A B⁺-Tree uses the same principle and normally stores records or record pointers in its leaves, making it especially suitable for range queries.

Figure 2.3 shows the central layout decision: one I/O loads an entire node and supplies many routing keys and pointers. The height, and therefore the number of block reads for a point query, is approximately

$$h = \log_B \left(\frac{n}{B} \right) = \frac{\log_2(n/B)}{\log_2 B}.$$



each node occupies approximately one physical block B

Figure 2.3: A block-oriented B-Tree with high fan-out.

More precisely, implementation details introduce ceilings and occupancy factors. The internal-node space is small compared with the leaves because

$$\text{internal nodes} \leq \frac{n}{B^2} + \frac{n}{B^3} + \cdots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Quantitative Comparison

Ignoring ceilings, the I/O speedup of a B-Tree over binary search is

$$\frac{\text{I/O}_{\text{binary search}}}{\text{I/O}_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Table 2.3: Illustrative comparison for a large ordered dataset.

Parameter	Value
Dataset size	$n = 2^{40}$ elements, approximately 10^{12} records
Block size	$B = 2^{14} = 16\,384$ records
Binary-search I/O	$40 - 14 = 26$ block accesses
B-Tree I/O	$(40 - 14)/14 \approx 1.85$, hence about 2 block accesses

For millisecond-scale storage latency, the difference between approximately 26 and 2 block accesses changes a point query from hundreds of milliseconds to only a few milliseconds, subject to caching and hardware details.

2.4 I/O-Aware Algorithms and Virtual Paging

There are two complementary ways to manage transfers between fast and slow memory:

I/O-aware design (explicit). The algorithm models B and the fast memory capacity M directly, and manages buffers, data layout, and transfers programmatically.

Virtual memory and paging (implicit). The operating system and the hardware transparently move pages using page tables and replacement policies such as LRU approximations.

2.4.1 Paging with an Oversized Working Set

Assume that physical memory has capacity M and is divided into M/B frames. If the application requires

$$n = (1 + \varepsilon)M = M + \varepsilon M, \quad \varepsilon > 0,$$

then at least εM units of the working set must reside in secondary storage. Let α be the fraction of instructions that perform memory operations. A typical instruction profile is:

- fraction $1 - \alpha$: arithmetic or logical operations, with cost approximately one CPU cycle;
- fraction α : ‘LOAD’ or ‘STORE’ operations;
- probability $1 - P(\varepsilon)$: the referenced data are resident in RAM, with access time t_m ;
- probability $P(\varepsilon)$: the access causes a page fault, with service time t_{disk} .

The resulting average time per instruction is

$$T_{\text{avg}} = (1 - \alpha) \cdot 1 + \alpha [(1 - P(\varepsilon))t_m + P(\varepsilon)t_{\text{disk}}].$$

Since $t_m \ll t_{\text{disk}}$, the page-fault term can dominate:

$$T_{\text{avg}} \approx \alpha P(\varepsilon) t_{\text{disk}}.$$

This approximation is a warning about sensitivity, not an identity: when $P(\varepsilon)$ is zero, the baseline CPU and RAM terms still matter.

2.4.2 Numerical Thrashing Example

Take $\alpha = 0.3$, a page-fault penalty of $t_{\text{disk}} = 10^5$ clock cycles, and a fault probability of only $P(\varepsilon) = 10^{-3}$. Then the expected penalty is

$$T_{\text{penalty}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cycles per instruction.}$$

One page fault every thousand memory accesses can therefore add about 30 cycles to every instruction on average. Relative to a one-cycle baseline, this can produce an order-of-magnitude slowdown, potentially in the $10\times$ – $30\times$ range depending on the ordinary RAM latency and the actual working-set behavior. When the fault rate rises further, the process enters *thrashing*: the system spends most of its time servicing page faults instead of progressing with useful computation.

2.5 Key Takeaways

1. A modular stride visits all logical blocks only when the stride and the number of logical blocks are coprime.
2. Block-aware layouts and access patterns can change I/O complexity by a factor of B , even when CPU work is unchanged.
3. B-Trees reduce external-memory search depth by matching their fan-out to the physical block size.
4. Hardware acceleration improves linear and polynomial algorithms in different ways, but gives only an additive improvement to exponential algorithms.
5. Virtual memory is convenient, but an oversized working set can turn a small page-fault probability into a dominant performance cost.

Chapter 3

Permutation, Sorting, and External Memory Merge Sort

Lecture 3

Permutation vs. Sorting and External Memory Merge Sort

Date: September 18, 2026

In this chapter, we examine the fundamental relationship between the permutation problem and the sorting problem in the two-level memory hierarchy. Unlike the classical RAM model, where permuting is asymptotically cheaper than sorting, in hierarchical memory systems the mechanics of block-based transfers impose structural constraints that intimately connect the two problems. Furthermore, we introduce and analyze the external Binary Merge Sort, highlighting its efficiency limitations and establishing the foundation for multiway merging.

3.1 The Permutation vs. Sorting Problem

3.1.1 Problem Formulation

We are given two vectors of size n :

- A data array $S[1 \dots n]$.
- A permutation vector $\pi[1 \dots n]$, containing a bijective permutation of the indices $\{1, 2, \dots, n\}$.

The objective is to rearrange the elements of S into a new array S' such that each original element $S[i]$ is placed into the destination position specified by $\pi[i]$:

$$S'[\pi[i]] = S[i], \quad \forall i \in \{1, \dots, n\}. \quad (3.1)$$

Example 3.1.1 (Index Permutation). Consider an array of 4 elements and its corresponding permutation vector:

$$\begin{aligned} \text{Indices: } & 1 \quad 2 \quad 3 \quad 4 \\ S = & [A, \quad B, \quad C, \quad D] \\ \pi = & [3, \quad 1, \quad 2, \quad 4] \end{aligned}$$

Applying the definition element by element:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

The resulting permuted array is therefore:

$$S' = [B, \quad C, \quad A, \quad D].$$

In Figure 3.1, the mapping from the source array to the destination array is illustrated visually.

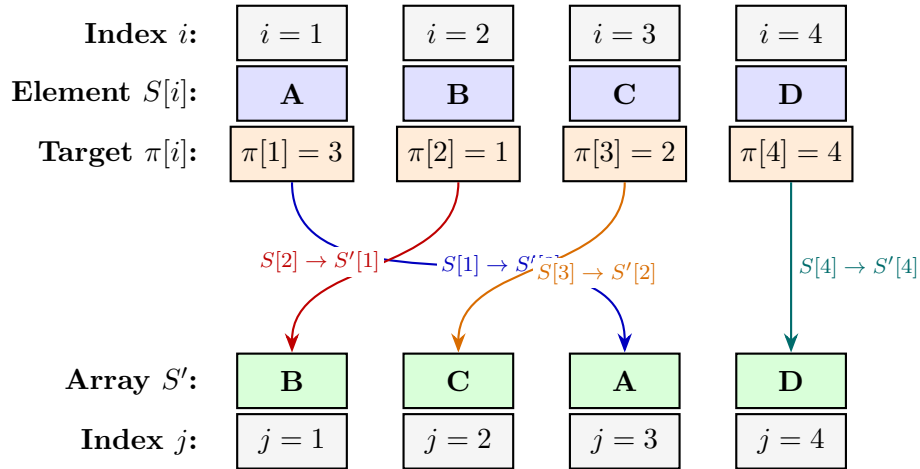


Figure 3.1: Permutation mapping: each element $S[i]$ is relocated to the destination index specified by $\pi[i]$ in array S' .

3.1.2 Naive Implementation on the RAM Model

In the standard RAM model, where any read or write access to arbitrary memory locations costs $\mathcal{O}(1)$ time, the permutation is implemented in a single linear pass:

Algorithm 1: Direct Permutation on RAM

Input: Array $S[1 \dots n]$, permutation vector $\pi[1 \dots n]$

Output: Permuted array S in place

```
1 for  $i \leftarrow 1$  to  $n$  do  
2    $S'[\pi[i]] \leftarrow S[i]$   
3  $S \leftarrow \text{copy}(S')$ 
```

The time complexity in the RAM model is strictly linear:

$$T_{\text{RAM}}(n) = \Theta(n).$$

Therefore, in internal memory, permuting is asymptotically faster than sorting by comparisons, which requires $\Omega(n \log n)$ operations.

3.1.3 Simulation Principle (RAM to Two-Level Memory)

Theorem 3.1.1 (Simulation Theorem). *Any sequential algorithm formulated for the RAM model with time complexity $T(n)$ incurs at most $\mathcal{O}(T(n))$ block transfers (I/Os) in the two-level memory model with parameters (M, B) .*

Proof Sketch. In the worst case, every single RAM operation accessing memory addresses a memory cell that resides in a block not currently resident in internal cache M . Each such access triggers a block transfer from disk, resulting in exactly 1 I/O operation. Multiplying the total step count $T(n)$ by the upper bound of 1 I/O per step yields the trivial bound of $\mathcal{O}(T(n))$ I/Os. $\square$

Although the simulation principle guarantees that any RAM algorithm can be ported to external memory, the bound $\mathcal{O}(T(n))$ is often thoroughly sub-optimal because it fails to exploit the block aggregation factor B .

3.1.4 The Relationship Between Sorting and Permutation

In the two-level memory hierarchy, the complexity relationship changes fundamentally, as summarized in [Table 3.1](#):

To understand this equivalence in block-based storage, consider the two reductions:

1. **Sorting via Permutation:** Given an array S , sorting requires computing the permutation π_S that puts keys in order, and then applying π_S to S .

Problem	RAM Model	Two-Level Memory (I/O)
Permutation Perm(n)	$\Theta(n)$	$\Theta\left(\min\left\{n, \text{Sort}(n, M, B)\right\}\right)$
Sorting Sort(n)	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Table 3.1: Asymptotic complexity comparison between RAM and External Memory models.

2. **Permuting via Sorting (Pair Reduction Algorithm):** Permuting an array S can be converted into an external sorting problem via the following three-step procedure:

- **Step 1 (Scan and Pair Creation):** Perform parallel linear scans over S and π to construct a set of n records containing $\langle S[i], \pi[i] \rangle$. Since this requires only streaming sequential I/O, the cost is:

$$\text{I/O}_{\text{Step 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

- **Step 2 (Sorting by Destination):** Sort the n records using the second component $\pi[i]$ as the sorting key. The cost is identical to external sorting:

$$\text{I/O}_{\text{Step 2}} = \text{Sort}(n, M, B).$$

- **Step 3 (Scan and Extraction):** Since the records are now sorted by increasing target index, a simple sequential scan extracts the first coordinate $S[i]$ into the destination array S' . The cost is:

$$\text{I/O}_{\text{Step 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

The overall I/O cost of this reduction is:

$$\text{I/O}_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \text{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\text{Sort}(n, M, B)). \quad (3.2)$$

Example 3.1.2 (Step-by-Step Pair Reduction Trace on Concrete Arrays). To visualize the reduction with concrete values, consider the instance from [Example 3.1.1](#):

$$S = [A, B, C, D], \quad \pi = [3, 1, 2, 4].$$

The algorithm executes across three sequential states of the record array:

- **Step 1 (Pairing):** Sequentially pair each item with its designated target index:

$$P = [\langle A, 3 \rangle, \langle B, 1 \rangle, \langle C, 2 \rangle, \langle D, 4 \rangle].$$

- **Step 2 (External Sorting on $\pi[i]$):** Sort the record array using the second coordinate (target destination) as the sorting key:

$$P_{\text{sorted}} = [\langle B, 1 \rangle, \langle C, 2 \rangle, \langle A, 3 \rangle, \langle D, 4 \rangle].$$

- **Step 3 (Sequential Extraction):** Stream through the sorted records and project their first coordinates to form the destination array:

$$S' = [B, C, A, D],$$

which correctly fulfills the required permutation since $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

The pair reduction pipeline is illustrated in [Figure 3.2](#).

3.2 Permutation Complexity in External Memory

The fundamental lower bound for permutation in the two-level memory model was established by Aggarwal and Vitter [2]:

$$\text{Perm}(n, M, B) = \Theta \left(\min \left\{ n, \frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right\} \right). \quad (3.3)$$

Letting $L = \log_{M/B}(n/B)$ denote the number of merge passes required by multiway merge sort:

$$\text{Sort}(n, M, B) = \Theta \left(\frac{n}{B} \cdot L \right). \quad (3.4)$$

3.2.1 Corner Case Analysis

Comparing the direct naive algorithm with the sorting-based reduction yields two distinct regimes determined by the relationship between block size B and pass count L :

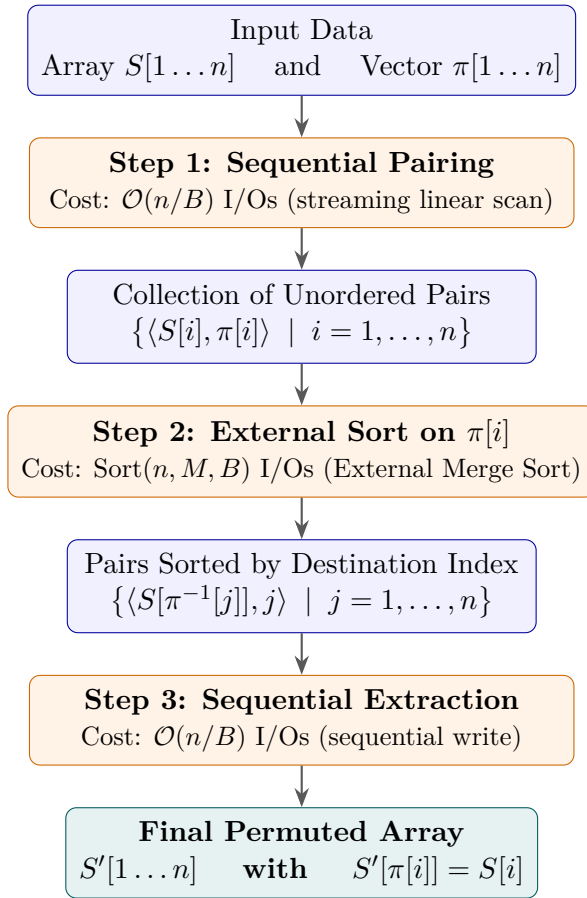


Figure 3.2: Operational workflow of the pair-reduction permutation algorithm.

- **Case 1** ($n \geq \frac{n}{B} \cdot L \iff B \geq L$): The block size B is greater than or equal to the depth of the merge tree. In this setting, sorting the pairs requires fewer I/Os than random single-element access:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Case 2** ($n < \frac{n}{B} \cdot L \iff B < L$): If the block size were pathologically small compared to the overall dataset ($B < L$), direct element-by-element random writing (at most n I/Os) would be asymptotically cheaper than sorting:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Numerical Verification on Realistic Architectural Parameters

To establish which regime governs real-world computing systems, consider extreme yet realistic parameters:

- Fast internal memory: $M = 16 \text{ GB} = 2^4 \cdot 2^{30} = 2^{34}$ elements.
- Disk block size: $B = 32 \text{ KB} = 2^5 \cdot 2^{10} = 2^{15}$ elements.
- Input dataset size: $n = 2^{60}$ elements (Exabyte scale).

We calculate the merge depth L :

$$L = \log_{M/B} \left(\frac{n}{M} \right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Substituting the exponential values:

$$\begin{aligned} \frac{n}{M} &= \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2 \left(\frac{n}{M} \right) = 26, \\ \frac{M}{B} &= \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2 \left(\frac{M}{B} \right) = 19. \end{aligned}$$

Hence:

$$L = \frac{26}{19} \approx 1.36 < 2.$$

Because $B = 2^{15} = 32\,768$ and $L \leq 2$, the condition $B \geq L$ holds by more than four orders of magnitude ($32\,768 \gg 2$). Consequently, in all practical systems, permutation is strictly bound by sorting complexity:

$$\text{Perm}(n) = \Theta(\text{Sort}(n)).$$

3.3 External Memory Binary Merge Sort

The classical recursive Merge Sort algorithm extends naturally to external memory by recursively dividing arrays, ordering runs, and merging sorted sub-arrays:

Algorithm 2: Binary Merge Sort

Input: Array S , boundaries i, j

Output: Sorted sub-array $S[i \dots j]$

```

1 if  $i < j$  then
2    $m \leftarrow \lfloor (i + j)/2 \rfloor$ 
3    $\text{MS}(S, i, m)$ 
4    $\text{MS}(S, m + 1, j)$ 
5    $\text{MERGE}(S, i, m, j)$ 

```

3.3.1 Buffer Architecture in the Merge Procedure

Given two consecutive sorted runs of length l , the merge procedure produces a single consolidated run of length $2l$ into an auxiliary array S' .

- **In the RAM Model:** The operation performs at most $2l$ comparisons and $2l$ memory writes, costing $\Theta(l)$ CPU steps.
- **In the Two-Level Model:** To minimize block transfers, internal memory M is partitioned into 3 dedicated buffers, each having size exactly equal to one block B :
 1. **Input Buffer 1:** holds one block (size B) of the first sorted run.
 2. **Input Buffer 2:** holds one block (size B) of the second sorted run.
 3. **Output Buffer:** accumulates merged elements in sorted order until full (B elements).

Figure 3.3 illustrates the internal memory allocation and streaming I/O pattern during the binary merge.

The execution protocol proceeds as follows:

1. The leading elements of the two input buffers are compared in CPU; the smaller key is appended to the output buffer.
2. Whenever an input buffer is exhausted, a disk read loads the next block of that run ($\frac{l}{B}$ I/Os per input run).

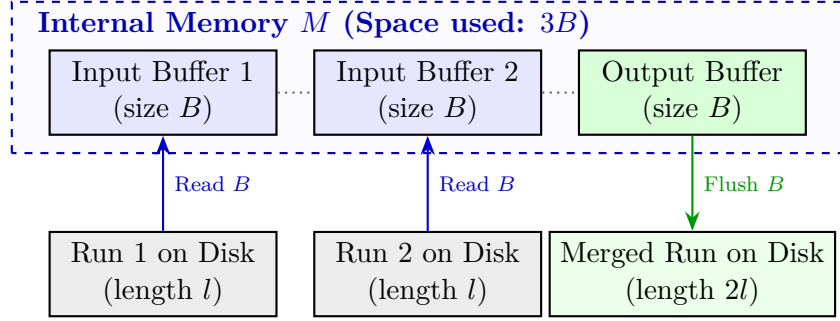


Figure 3.3: Internal memory buffer organization during binary external merge.

3. As soon as the output buffer fills up with B items, it is flushed to disk in a single write operation ($\frac{2l}{B}$ write I/Os overall).

The total I/O cost of merging two runs of length l is:

$$\text{I/O}_{\text{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \quad (3.5)$$

3.3.2 Recursion Tree Complexity

Recursion terminates once sub-array sizes fit entirely within internal memory M . At this base level, each chunk of size M is read into memory, sorted using an in-memory algorithm (e.g., QuickSort or HeapSort) with zero additional I/O, and written back to disk using $\mathcal{O}(M/B)$ I/Os.

This yields:

$$N_{\text{run}} = \frac{n}{M} \text{ initial sorted runs of size } M.$$

The binary merge tree has height:

$$h = \log_2\left(\frac{n}{M}\right) \text{ levels.} \quad (3.6)$$

Figure 3.4 depicts the recursion structure of binary external merge sort.

At each level of the tree, all n elements are streamed exactly once, requiring $\Theta(n/B)$ I/Os per level. Multiplying by tree height gives the overall complexity:

$$\text{I/O}_{\text{BinaryMergeSort}} = \mathcal{O}\left(\frac{n}{B} \log_2\left(\frac{n}{M}\right)\right). \quad (3.7)$$

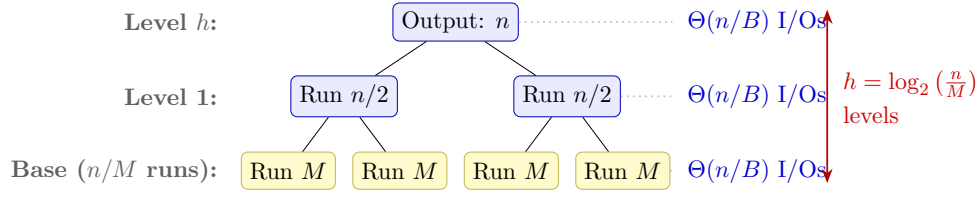


Figure 3.4: Recursion tree of external memory Binary Merge Sort with level I/O costs.

3.3.3 Sub-optimality and Bottlenecks

Although significantly more efficient than naive approaches, this design suffers from a major structural bottleneck:

- **Drastic internal memory underutilization:** During merging, only 3 blocks ($3B$ elements) are occupied in fast memory M . The remaining capacity ($M - 3B$) sits completely idle.
- **Fixed logarithm base:** The base of the logarithm is fixed at 2, whereas memory M could accommodate up to:

$$k \approx \frac{M}{B} - 1 \text{ concurrent input streams.}$$

- **Loss of the $\log_2(M/B)$ convergence factor:** By failing to perform k -way merging (*Multiway Merge Sort*), the algorithm executes substantially more passes, falling short of the optimal lower bound:

$$\mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right).$$

Figure 3.5 visually contrasts this internal memory underutilization against full buffer saturation.

The study of Multiway Merge Sort and the elimination of this bottleneck form the focus of subsequent lectures.

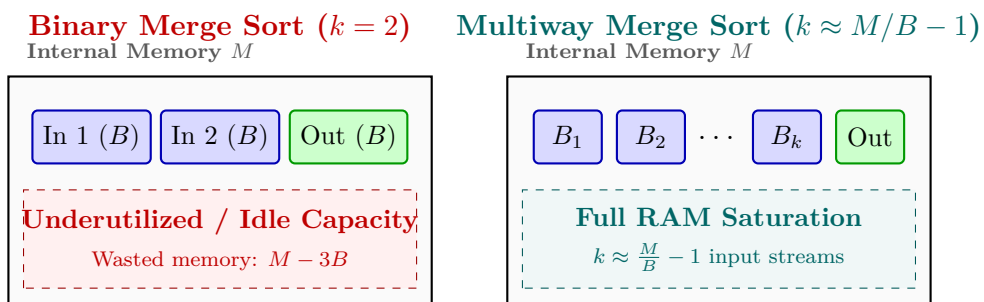


Figure 3.5: Internal memory allocation efficiency: severe underutilization in Binary Merge Sort versus full buffer saturation in Multiway Merge Sort.

Bibliography

- [1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.
- [2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.